

Foundation of Blockchain Technology (IS501) Pre-Reading Material – Lecture 23

Attacks on Smart Contract

Estimated Reading Time: 15–20 minutes

Introduction

A **smart contract** is a program deployed on a blockchain that automatically executes predefined rules. Since smart contracts may control digital assets and execute transactions automatically, security vulnerabilities can have serious consequences.

Unlike traditional software, blockchain transactions are generally difficult to reverse. Therefore, **secure coding and testing are essential before deploying a smart contract.**

Why Are Smart Contracts Vulnerable?

Smart contracts can be attacked because of:

- Programming errors and logical flaws
- Improper access control
- Unsafe external calls
- Poor input validation
- Weak randomness
- Manipulation of external data
- Incorrect assumptions about blockchain behavior

A small coding mistake can potentially allow an attacker to manipulate contract behavior or gain unauthorized access to assets.

Common Smart Contract Attacks

Reentrancy Attack

An attacker repeatedly calls a vulnerable contract function before the previous execution has completed. This can result in unauthorized withdrawal of funds.



Prevention: Follow the **Checks–Effects–Interactions** pattern and use appropriate reentrancy protection.

Access Control Attack

A sensitive function is accidentally made accessible to unauthorized users. An attacker may then change contract settings, transfer ownership, or withdraw funds.

Prevention: Use proper access modifiers and carefully verify permissions.

Integer Overflow/Underflow

An arithmetic operation produces a value outside the expected numerical range. Modern Solidity versions provide built-in protection in normal arithmetic operations, but developers must still understand this vulnerability.

Denial-of-Service (DoS)

An attacker causes a contract function to fail or become unusable, for example by exploiting problematic external calls or unbounded operations.

Oracle Manipulation

A smart contract depends on external information such as cryptocurrency prices. If the data source can be manipulated, an attacker may exploit the incorrect information.

Front-Running

An attacker observes a pending transaction and submits another transaction designed to benefit from it before the original transaction is confirmed. This is closely related to **Maximum Extractable Value (MEV)**.

Preventing Smart Contract Attacks

Developers should:

- Follow secure coding practices.
- Implement proper access control.
- Validate user inputs.
- Minimize unnecessary external calls.
- Avoid unbounded loops.
- Use reliable oracle mechanisms.

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



- Test contracts with different inputs and conditions.
- Perform code reviews and security audits.
- Use static analysis and fuzz testing tools.

Key Takeaway

Smart contract security is essential because blockchain transactions are automated, public, and generally difficult to reverse.

Understanding common attacks such as **reentrancy, access-control vulnerabilities, DoS, oracle manipulation, and front-running** helps developers design safer blockchain applications.

Think Before Class

If a smart contract has a programming vulnerability and an attacker exploits it, should the problem be considered a coding error, a design error, or a security failure? Why?

